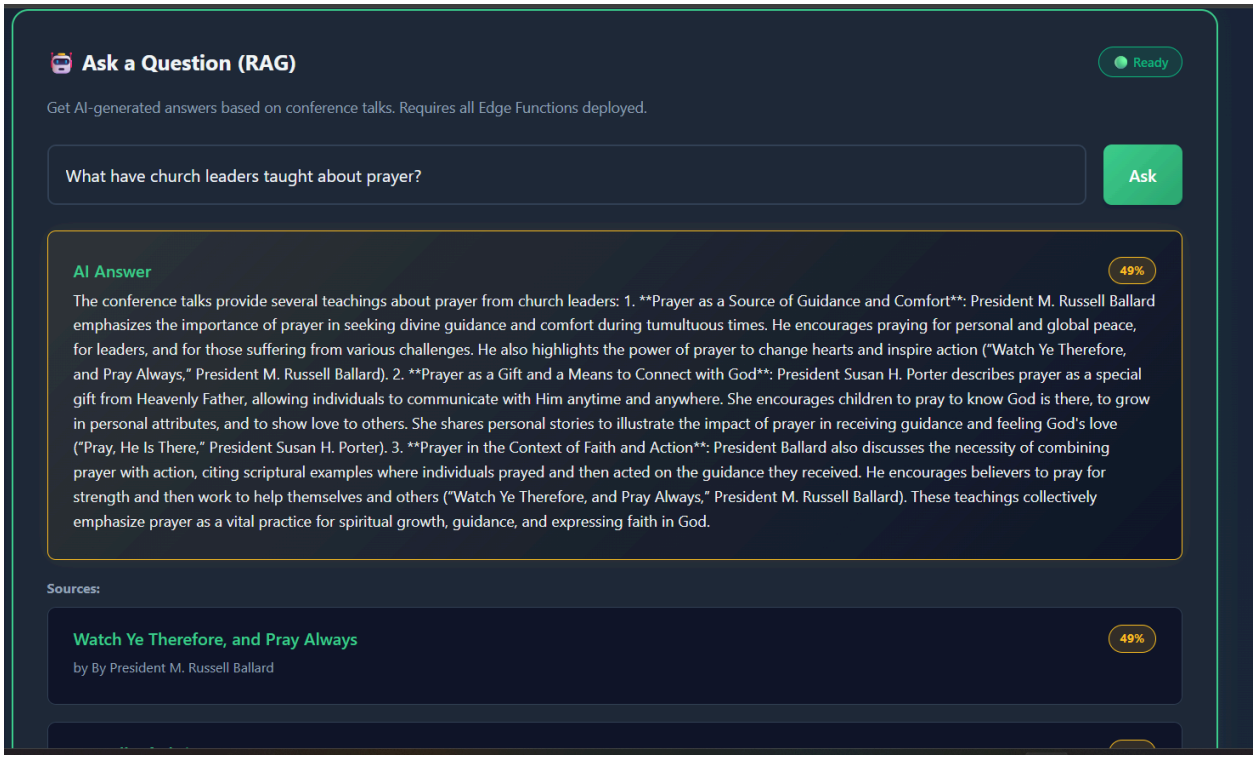
Cecily Black

Conference RAG

GitHub Repository URL: https://github.com/ZonkeyZebra/conference-rag

Live Deployment URL: https://zonkeyzebra.github.io/conference-rag/

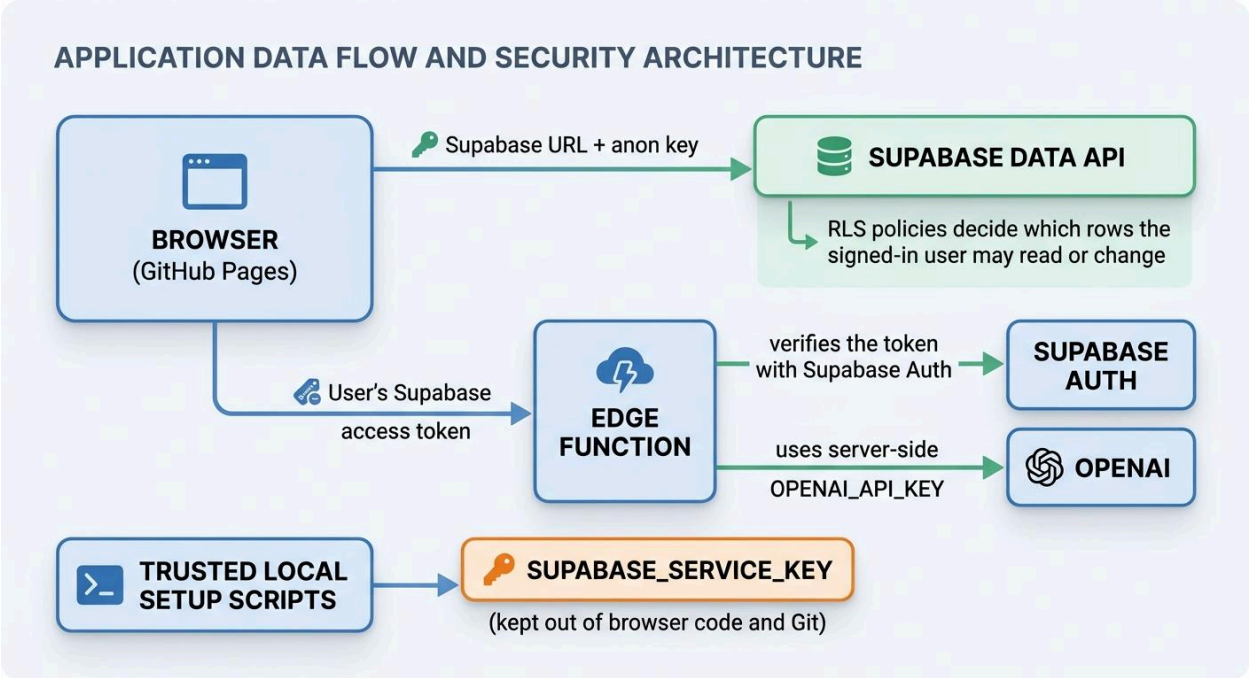
Screenshot Working RAG Query + Answer:



Custom Feature: I decided to add a topic/theme analyzer for each General Conference. A user is able to select the year and and month and they are given the top three topics that came up during that conference with some talks that are examples of that topic being discussed with a link to the talk. I selected this feature as I thought it would be interesting to see how using embeddings/AI could help to identify common themes between all the talks for each conference.

Reflection

Security Architecture:



The anon key is intended to identify the Supabase project in client-side applications, so it is expected to be visible in config.public.json and in browser network requests. It is not a password and does not, by itself, grant unrestricted access. This is why the anon key is safe to expose in config.public.json.

The service role key, on the other hand, would cause quite a few issues if someone got a hold of that. This key has privileges that would let someone read, modify, or delete the database data. So, it's important to keep this key hidden.

The Edge Functions allow the browser to call the OpenAI API without revealing the key and allows it to stay secret. The functions read OPENAI_API_KEY from server-side secrets, so the key is not sent to the browser. The function also checks the caller's bearer token before using the key.

RLS is the database-enforced authorization layer. It applies policies to rows even when a client connects directly to the Data API, rather than relying only on the app to hide or filter data. RLS is only as protective as its policies: any rows explicitly allowed to the anon role are public to anyone with the project URL and anon key.

Edge Functions & the "Secure Middle":
Calling OpenAI directly from a browser would expose the OpenAI API key in downloadable JavaScript or network requests. Anyone could copy it and use the

account's quota and billing. A browser is a public environment, so secrets cannot be protected there.

To prevent the secret OpenAI API key from being revealed, we use this flow instead:
1. The signed-in browser invokes an Edge Function with the question and its Supabase access token.
2. The function checks the bearer token with Supabase Auth. This verifies that the request is associated with a valid signed-in user; it is not the OpenAI key and does not replace database RLS.
3. If verification succeeds, the function reads the private OpenAI key from its server-side environment and requests an embedding or generated answer.
4. The function returns only the result needed by the app.

The same pattern is useful whenever a public client needs a private capability: for example, a backend can validate a user, enforce input and usage limits, and call a payment, email, maps, or AI provider without revealing that provider's secret key. Authentication identifies the caller; the server must still authorize the requested action and manage abuse and costs.

From SQL to Semantics:
SQL ILIKE is best for the keyword search since it can find text with the entered words or phrases quickly and get consistent results. It would not work well for the semantic or RAG searches though, as it would miss anything that does not have the exact wording provided in the query.

A semantic search turns text into embeddings, which are vectors of numbers representing features learned from language data. Vectors with similar directions are treated as semantically related. This works better for finding sentences that would be related to the query provided by the user.

As an example that can compare a ILIKE and semantic search, keyword search for "peace during hard times" may miss a sentence that says "the Savior can calm our hearts in seasons of trial" because the exact words differ. Semantic search may retrieve it because the phrasing is related in meaning. Conversely, if I need a passage containing a specific phrase such as "faith as small as a mustard seed," ILIKE is better for finding that exact wording; semantic search may return generally related passages that do not contain the quote.

RAG vs Fine-Tuning:

Both RAG and fine-tuning are ways to make an AI model more useful at accomplishing a particular task. The biggest difference is whether you give the model relevant information when it answers or change how the model itself behaves. RAG gives the model relevant information from an external knowledge source at question time. Whereas, fine-tuning trains an existing model further on examples so it learns a particular style, task, or pattern of behavior.

This means that RAG has less training overhead and always has current information, but may only have limited context and will be more variable in response. Fine-tuning will produce a far more reliable and reproducible output, but requires more training costs and maintenance as it may hold onto outdated information.

RAG is a better choice for this project since there are many talks with the potential of more information being added to the database every six months. The queries about conference talks should also remain simple enough that needing to train the model for more consistent results would not be all that helpful or beneficial.

Fine-tuning could be a better choice when the main goal is consistent behavior, tone, or output structure across many requests, especially when there are many high-quality examples of the desired behavior. It is usually not the best way to make a model reliably recall and cite a changing collection of detailed talks: updates require more training, and the model's learned parameters do not provide the same direct source references as retrieval.

AI-Assisted Development:
My agent was really good at walking me through steps and helping me to complete and generate edge functions for this project. There were some things that were explained on the GitHub walkthrough for the project that AI was able to expand upon and explain. It also helped me a lot with the debugging process and helped me realize that I would need to add an index table to help with the timeout errors my application was experiencing.

I needed to guide it on where to ask me questions to help my understanding and on my confusion with the timeout errors I was experiencing. I wasn't convinced that was the correct solution at first, so I had to do some more debugging with it before I could more clearly understand the issue and why it was giving me the solution it was.

I learned that AI and agents are super helpful at generating functions to get things done, but can have a hard time helping with debugging when it is not something it can run and test for itself. I also learned that it is really helpful to have agents check my

understanding and ask me questions as I was trying to understand the material and why it was doing the things it was in order to finish the project.